



CMPE 482 – Robotics

Spring 2026

Final Project

Autonomous Warehouse Robot Simulation with Route Optimization and Path Planning

Submitted By:

Naim Ashour ID: A00957

Praharsh ID: A00490

Instructor:

Dr. Rabé Andersson

Date Submitted:

4/5/2026

Table of Contents

1. Introduction	3
1.1 Project Title	3
1.2 Problem Statement:	3
1.3 Objectives:	3
1.4 Tools and Technologies Used:	4
2. Design	4
2.1 System Architecture / Block Diagram:	4
2.2 Design Methodology:	6
2.3 Component Description:	6
3. Implementation	7
3.1 Microcontroller Code:	7
3.2 Simulation and Compilation:	9
4. Testing and Results	10
4.1 Test Plan:	10
4.2 Embedded Systems Demonstration:	11
4.3 Observations and Results:	12
5. Conclusions and Future Work	14
5.1 Key Learnings:	14
5.2 Challenges Encountered:	15
5.3 Recommendations for Future Improvement:	16
6. Team Contributions	17
7. References	17

1. Introduction

1.1 Project Title:

Autonomous Warehouse Robot Simulation with Route Optimization and Path Planning

1.2 Problem Statement:

Warehouses today need to work faster and more efficiently, especially with the increase in online shopping. Many companies use robots to move items between different areas like storage, packing, and delivery. This helps reduce time and effort, but it also creates a challenge. The robot needs to handle multiple delivery tasks and still move safely without making mistakes.

One of the main problems is deciding the order of tasks. A simple approach is to always go to the nearest location, but this is not always the best solution. In some cases, the robot may end up traveling a longer total distance, which wastes time and reduces efficiency. So, finding a better order for completing tasks becomes important.

Another problem is navigation. The robot has to move in an environment that has obstacles like shelves or walls. If the robot does not plan its path properly, it might take longer routes or even crash into obstacles. This means we need a method that can help the robot move safely while still reaching its destination in a good way.

In this project, we try to solve both of these problems by building a simulation of a warehouse robot. The robot is designed to choose a better order for completing tasks and then use the A* algorithm to find safe paths between different locations. The environment is set up as a grid, where some cells are blocked by obstacles. Even though this is a simulation, it gives a good idea of how similar problems are handled in real warehouse systems.

1.3 Objectives:

The main objective of this project is to design and implement a simulated autonomous robot that can operate in a warehouse-like environment. The robot should be able to handle multiple delivery tasks while deciding the order in which they are completed. It also needs to move safely by avoiding obstacles and following planned paths. By combining task scheduling with path planning, the project shows how robots can complete tasks more efficiently.

Specifically, the project aims to:

- Simulate a robot in a grid-based environment

- Implement task sequencing (greedy and optimized)
- Use A* for collision-free path planning
- Convert planned paths into robot movement
- Evaluate performance using path cost

1.4 Tools and Technologies Used:

This project was developed using a combination of simulation tools and programming to model and test the behaviour of an autonomous warehouse robot. The system combines task scheduling, path planning, and movement control in a simulated environment, allowing the robot to operate without physical hardware. This makes it easier to test different approaches and compare their performance.

The main tools and technologies used in this project include:

- **Webots Simulation Environment:** Used to design the warehouse setup and simulate the robot's movement and interaction with obstacles
- **Python Programming Language:** Used to implement the system logic, including path planning and task sequencing
- **Grid-Based Environment:** Used to represent the warehouse layout in a simple and structured way
- *A Path Planning Algorithm:** Used to compute efficient and collision-free paths
- **Task Scheduling Methods:** Includes greedy and optimized approaches for deciding delivery order

2. Design

2.1 System Architecture / Block Diagram:

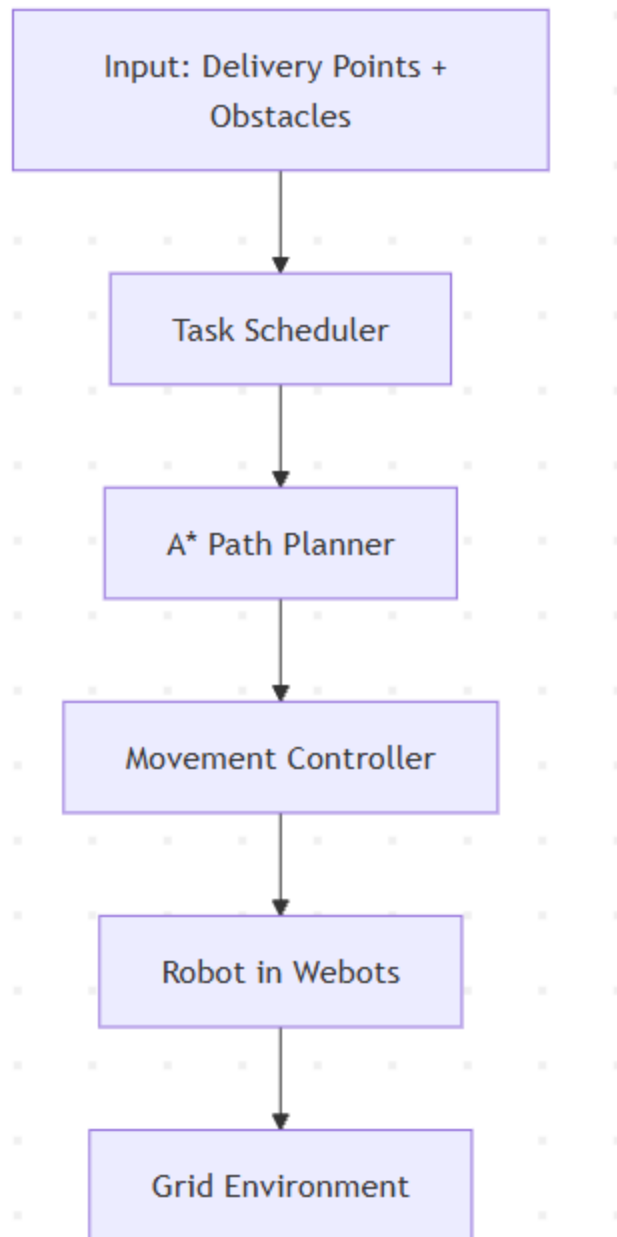


Figure 2.1: System architecture of the autonomous warehouse robot simulation

The diagram shows the overall structure of the system and how each part is connected. The process starts by defining the delivery points and obstacles in the environment. These are passed to the task scheduler, which decides the order in which the robot should complete the tasks.

After that, the A* path planner is used to compute a safe path between the selected locations while avoiding obstacles. The generated path is then sent to the movement controller, which converts it into simple actions such as moving forward and turning.

Finally, these commands are executed by the robot in the Webots simulation. The robot moves within a grid-based environment that represents the warehouse and includes obstacles

2.2 Design Methodology:

The system was developed step by step to make sure each part works correctly. First, a grid-based environment was created to represent the warehouse. Obstacles were placed in specific positions, and delivery points were defined for the robot to visit.

Next, task scheduling was implemented. Two approaches were used: a simple greedy method and an optimized method. These methods decide the order in which the robot should complete the delivery tasks.

After that, the A* algorithm was implemented to find safe paths between the selected points. The algorithm avoids obstacles and generates a path as a sequence of grid positions.

Once the path is generated, it is converted into robot movement. This includes moving forward and turning based on the direction of the next step.

Finally, the system was tested in the Webots simulation to check if the robot can follow the path correctly and complete all task

2.3 Component Description:

The system is made up of several main components that work together to allow the robot to complete delivery tasks efficiently and safely. Each component has a specific role, and together they form the complete workflow of the system.

- **Environment:**
The grid environment represents the warehouse layout in a simplified way. It is modeled as a 2D grid where each cell can either be free space or an obstacle. Obstacles represent items such as shelves or walls that the robot cannot pass through. This structure makes it easier to apply path planning algorithms and test different navigation scenarios.
- **Task Scheduler**
The task scheduler is responsible for deciding the order in which the robot visits the delivery points. In this project, two methods are used: a greedy approach and an optimized approach. The greedy method selects the nearest next point, while the optimized method considers different possible orders to reduce the total path cost. This component directly affects how efficient the overall system is.
- *A Path Planner:**
The A* algorithm is used to find the shortest path between two points in the grid while avoiding obstacles. It works by combining the actual distance traveled with an estimated distance to the goal. The result is a sequence of grid positions that form a safe and efficient path. This component is essential for ensuring that the robot does not collide with obstacles.
- **Movement Controller:**
The movement controller takes the path generated by the A* algorithm and converts it into actions that the robot can perform. These actions include moving forward and turning left or right depending on the direction of the next step. Since the system uses timed motor control, this component is important for making sure the robot follows the path as closely as possible.
- **Robot (E-puck Model):**
The robot used in this project is the E-puck model in the Webots simulator. It acts as the physical agent that executes the commands generated by the system. The robot moves step by step through the grid environment, following the planned path and completing the delivery tasks.

3. Implementation

3.1 Microcontroller Code:

The robot control logic was implemented in Python using the Webots simulation environment. The code handles task scheduling, path planning, and robot movement. The robot follows paths generated by the A* algorithm and converts them into movement commands such as forward motion and turning.

The following code snippets show some of the main parts of the implementation.

Snippet 1: Environment Setup

```
25
26 # --- Environment Setup ---
27 grid = [[0]*10 for _ in range(10)]
28
29 obstacles = [(4,3), (4,4), (4,5), (4,6), (2,6), (3,6)]
30 for r, c in obstacles:
31     grid[r][c] = 1
32
33 start_pos = (0, 0)
34 stations = [(2, 2), (8, 2), (5, 8)]
```

This section creates the grid-based warehouse environment used in the simulation. Obstacles are placed in blocked cells, while the robot start position and delivery stations are also defined.

Snippet 2: A Path Planning*

```
for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
    neighbor = (current[0]+dr, current[1]+dc)

    if 0 <= neighbor[0] < 10 and 0 <= neighbor[1] < 10 and grid[neighbor[0]][neighbor[1]] == 0:
        tentative_g = g_score[current] + 1
```

This part is used by the A* algorithm to explore neighboring cells while searching for a safe path. The algorithm checks valid positions and avoids blocked cells that contain obstacles

Snippet 3: Routing Comparison

```

84
85 for p in itertools.permutations(stations):
86     cost, _ = get_total_path([start_pos] + list(p), grid)
87     if cost < best_cost:
88         best_cost = cost
89         best_order = [start_pos] + list(p)

```

This section tests different delivery orders to find the route with the lowest total path cost. The optimized routing method compares possible combinations and selects the most efficient one.

Snippet 4: Robot Movement Control

```

if target_dir != current_dir:
    cp = current_dir[0]*target_dir[1] - current_dir[1]*target_dir[0]
    dot = current_dir[0]*target_dir[0] + current_dir[1]*target_dir[1]

    if dot == -1:
        move_robot(TURN_180_DURATION, MAX_SPEED, -MAX_SPEED)
    elif cp == 1:
        move_robot(TURN_90_DURATION, MAX_SPEED, -MAX_SPEED)
    elif cp == -1:
        move_robot(TURN_90_DURATION, -MAX_SPEED, MAX_SPEED)

move_robot(MOVE_DURATION, MAX_SPEED, MAX_SPEED)

```

This section controls the robot's movement and turning behavior. The code first checks whether the robot needs to change direction before moving to the next step in the path. Based on the direction difference, the robot performs either a 90-degree turn or a 180-degree turn by adjusting the wheel motor speeds. After the turn is completed, the robot moves forward to continue following the planned path.

3.2 Simulation and Compilation:

The project was developed and tested using the Webots robot simulator. The simulation environment was designed to represent a simple warehouse layout using a 2D grid. Obstacles were placed in different positions to act as shelves or blocked areas, while delivery points were added for the robot to visit.

The E-puck robot model was used in the simulation and connected to a Python controller file. The controller contains the main system logic, including task scheduling, A* path planning, and robot movement. After assigning the controller to the robot in Webots, the simulation was executed directly through the software.

During execution, the robot first calculated the delivery order and then generated a collision-free path between the different locations. The generated path was converted into movement commands such as moving forward and turning left or right.

The robot then followed these commands step by step inside the environment. The console output was used to monitor the behaviour of the system during simulation. It displayed information such as the selected delivery order, path cost, generated path coordinates, and movement steps followed by the robot. This made it easier to check if the robot was moving correctly and following the planned route.

Several tests were carried out using different obstacle placements and delivery point configurations. These tests helped evaluate how the robot behaved in different situations and allowed comparison between the greedy and optimized routing approaches.

Small movement inaccuracies were also observed during testing because the system uses timed motor control instead of sensor feedback, but the overall path planning logic worked correctly.

4. Testing and Results

4.1 Test Plan:

Several test cases were carried out to evaluate the behaviour, accuracy, and overall performance of the robot system. Different warehouse layouts were created by changing the obstacle positions and delivery point locations inside the grid environment. These tests were used to observe how the robot planned paths, selected delivery orders, and moved through the environment.

The first set of tests focused on the A* path planning algorithm. The robot was given a start position and a destination while obstacles were placed between them. The expected result was for the algorithm to generate a valid collision-free path that avoids blocked cells and safely reaches the target location.

Another important test involved task scheduling and routing. Multiple delivery stations were added to the environment, and both the greedy and optimized routing methods were tested. The purpose of this test was to compare the total path cost and observe how the selected delivery order affects the efficiency of the system. The expected result was for the optimized method to produce a lower or equal path cost compared to the greedy approach.

Movement testing was also performed to check if the robot could correctly follow the generated path inside the simulation. The robot was tested with straight movement, left turns, right turns, and longer navigation sequences through the warehouse environment. The expected result was for the robot to complete the route while maintaining the correct movement direction.

Additional tests were carried out using different obstacle arrangements to see how the system behaves in more difficult navigation scenarios. These tests helped verify that the robot could still find valid paths even when some routes were blocked.

The results from these tests were observed through the robot movement in Webots and through the console output, which displayed the generated paths, movement steps, and total path costs during execution.

4.2 Systems Demonstration:

The system demonstration was performed using the Webots simulation environment with the E-puck robot model. A warehouse-like environment was created using a 2D grid structure where some cells represented free space and others represented obstacles such as shelves or blocked areas. Delivery stations were also placed at different locations in the environment for the robot to visit.

At the beginning of the simulation, the robot was assigned multiple delivery points. The system first determined the order in which these tasks should be completed using the implemented routing logic. Both greedy and optimized routing approaches were tested during different runs of the simulation to compare their behaviour and overall path cost.

After selecting the delivery order, the A* path planning algorithm generated a collision-free path between the robot's current position and the target destination. The generated path consisted of a sequence of grid positions that the robot needed to follow while avoiding obstacles in the environment.

The movement controller then converted the generated path into robot actions such as moving forward, turning left, and turning right. As the simulation executed, the robot moved step by step through the warehouse environment while following the planned route. The robot navigation could be visually observed inside the Webots simulation window.

The console output was also used during the demonstration to monitor the internal behaviour of the system. Important information such as the selected delivery order, path cost, generated coordinates, and movement steps was displayed during execution. This helped verify that the robot was following the correct path and that the routing and planning algorithms were working as expected.

Several simulation scenarios were tested by changing the obstacle placements and delivery station positions. These tests allowed observation of how the robot adapts its route depending on the environment configuration. In some cases, the robot followed long straight paths, while in other cases it needed to perform multiple turns to navigate around blocked areas.

During testing, small movement inaccuracies were observed because the robot movement depended on timed motor control rather than sensor-based correction. This occasionally caused slight drifting or imperfect alignment with the grid. However, the overall navigation logic, path generation, and task scheduling worked successfully throughout the demonstration.

The demonstration successfully showed the integration of task scheduling, path planning, obstacle handling, and robot navigation within a simulated warehouse environment.

4.3 Observations and Results:

The simulation results showed that the robot was able to move through the warehouse environment and follow the generated paths successfully. The A* algorithm generated valid collision-free paths between the delivery stations while avoiding the blocked cells representing obstacles.

During execution, the robot followed the generated route step by step inside the Webots environment. The movement sequence included straight movement as well as left and right turns depending on the direction of the next path position. The robot

was generally able to reach the required delivery points while following the planned navigation path.

The console output shown in Figure 4.2 displays important information generated during execution. This includes the selected delivery order, total path cost, full generated path, and the movement steps followed by the robot. Both the greedy and optimized routing methods produced a total path cost of 19 in this test scenario. Although the results were equal in this case, the optimized approach is expected to perform better in larger and more complex warehouse layouts because it evaluates multiple possible delivery sequences before selecting the best route.

Several simulation tests were performed using different obstacle layouts and station positions. These tests helped evaluate how the robot behaved in different navigation situations. In most cases, the robot was able to avoid obstacles and continue moving toward the target locations successfully.

During testing, some small movement inaccuracies were observed. Since the robot movement depended on timed motor control rather than sensor feedback, slight drifting and imperfect turning alignment occasionally occurred. This was especially noticeable during longer movement sequences or repeated turns. To reduce this issue, movement and turning durations were adjusted several times during testing until smoother movement was achieved.

Figure 4.1 shows the robot navigating through the warehouse environment during simulation. The overall results demonstrate that the system was able to combine task scheduling, A* path planning, obstacle handling, and robot movement successfully within the simulated warehouse environment.

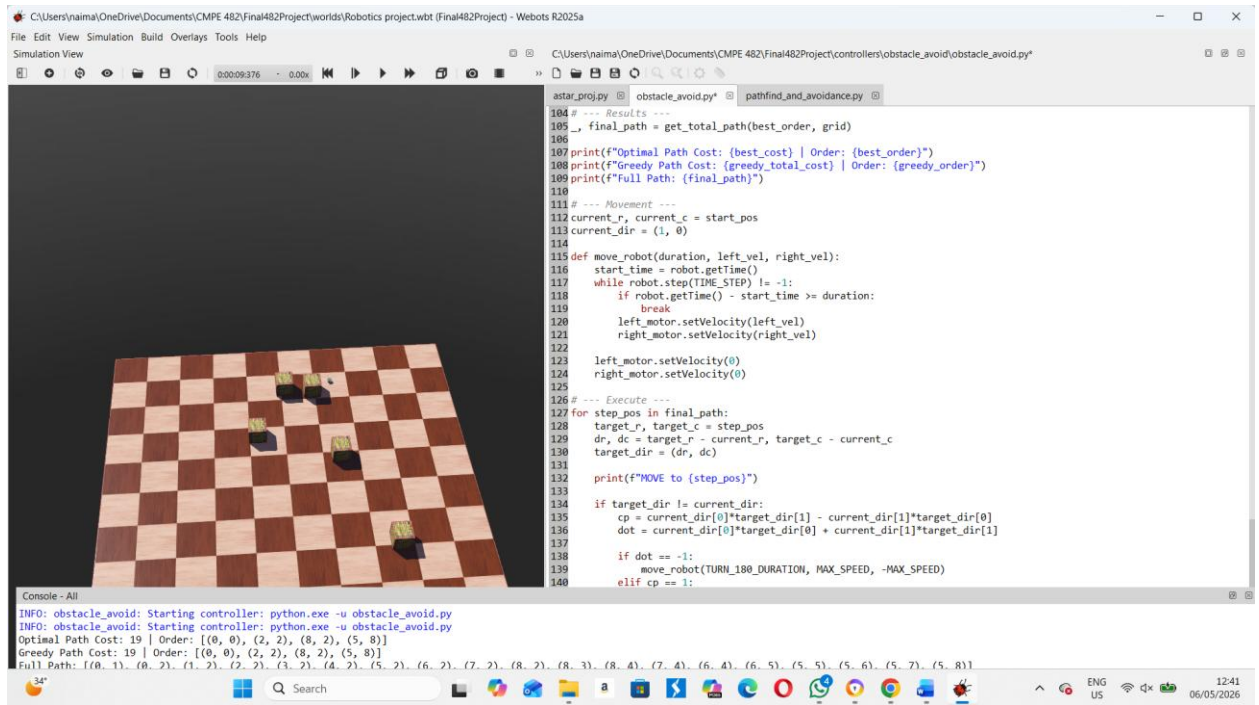


Figure 1: Robot Movement and path planning

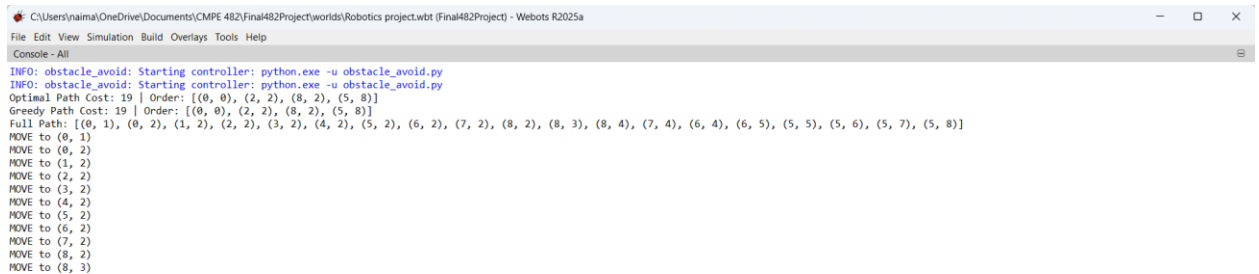


Figure 2: Console results through running simulation

5. Conclusions and Future Work

5.1 Key Learnings:

This project provided a better understanding of how autonomous robot systems combine task scheduling, path planning, and movement control to perform navigation tasks inside a warehouse environment. Throughout the project, different concepts related to robotics and simulation were applied in a practical way rather than only studying them theoretically.

One of the main things learned during the project was how the A* algorithm works in grid-based navigation. The project showed how path planning algorithms can generate safe and efficient routes while avoiding obstacles. It also demonstrated how routing decisions can affect the overall efficiency of the system when multiple delivery tasks are involved.

Another important learning outcome was understanding the relationship between software logic and robot movement. Even when the generated path is correct, the physical movement of the robot still depends on accurate timing and control. Small timing errors during movement and turning can affect the final robot position, which made testing and parameter adjustment an important part of the implementation process.

The project also improved skills in Python programming, simulation design, debugging, and testing inside the Webots environment. Different obstacle layouts and delivery scenarios were tested, which helped improve understanding of how autonomous navigation systems behave under different conditions.

Overall, the project provided practical experience in developing and testing a complete robotic simulation system from planning and design to implementation and evaluation.

5.2 Challenges Encountered:

Several challenges were encountered during the development and testing of the project. One of the main difficulties was controlling the robot movement accurately inside the simulation environment. Since the system relied on timed motor control rather than sensor feedback, small timing inaccuracies sometimes caused the robot to drift slightly or perform imperfect turns.

Another challenge involved aligning the robot movement with the grid system. During early testing, the robot occasionally moved outside the expected path or turned in the wrong direction because of incorrect orientation handling and movement timing values. Multiple adjustments were required to improve movement behaviour and make the navigation more stable.

Path planning and routing integration also required careful testing. The generated path from the A* algorithm needed to match the robot movement logic correctly so that the robot could follow the route step by step without skipping positions or colliding with obstacles.

In addition, different obstacle layouts sometimes produced unexpected movement behaviour, especially in narrow areas where turning precision became more important. These issues were addressed through repeated testing, debugging, and adjustment of movement durations and turning values until smoother behaviour was achieved.

Despite these challenges, the final system was able to demonstrate successful navigation, routing, and obstacle avoidance inside the warehouse simulation.

5.3 Recommendations for Future Improvement:

Several improvements can be made in future versions of the project to increase the accuracy and realism of the system. One possible improvement would be adding sensor-based feedback instead of relying only on timed motor control. This would help the robot correct its position more accurately and reduce movement drift during navigation.

Another improvement would be implementing dynamic obstacle handling, where the robot can detect changes in the environment and recalculate paths automatically during movement. This would make the system more suitable for realistic warehouse scenarios where obstacles and traffic conditions may change over time.

The project could also be expanded by using a larger warehouse environment with more delivery stations and multiple robots working together. This would allow more advanced routing and coordination strategies to be tested.

Additional improvements may include using machine learning techniques for smarter task scheduling, improving movement precision, and integrating more realistic warehouse models inside the simulation environment.

Overall, the project provides a strong foundation that can be expanded into a more advanced autonomous warehouse robot system in the future.

6. Team Contributions

Team Member	Student ID	Main Contributions
Naim Ashour	A00957	Developed the robot movement and navigation logic, implemented the A* path planning algorithm, designed the simulation environment in Webots, tested robot movement behaviour, and contributed to the report writing and presentation preparation .
Praharsh	A00490	Worked on task scheduling and routing logic, assisted in simulation setup and debugging, tested different obstacle and delivery scenarios, analyzed simulation results, and contributed to the report documentation and presentation slides.

7. References

[1] Cyberbotics Ltd., "Webots User Guide," Webots Documentation, 2025. [Online]. Available: <https://cyberbotics.com/doc/guide/index>

[2] Cyberbotics Ltd., "E-puck Robot Reference," Webots Documentation, 2025. [Online]. Available: <https://cyberbotics.com/doc/guide/epuck>

[3] Python Software Foundation, "Python 3 Documentation," Python.org, 2025. [Online]. Available: <https://docs.python.org/3/>

[4] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021. [Online]. Available: <https://aima.cs.berkeley.edu/>

[5] P. Corke, Robotics, Vision and Control: Fundamental Algorithms in MATLAB®, 2nd ed. Cham, Switzerland: Springer, 2017. [Online]. Available: <https://petercorke.com/rvc2/>

[6] M. Ben-Ari and F. Mondada, Elements of Robotics. Cham, Switzerland: Springer, 2018.
[Online]. Available: <https://link.springer.com/book/10.1007/978-3-319-62533-1>

[7] GeeksforGeeks, "A* Search Algorithm," 2025. [Online]. Available:
<https://www.geeksforgeeks.org/a-search-algorithm/>

[8] Red Blob Games, "Introduction to the A* Pathfinding Algorithm," 2025. [Online]. Available:
<https://www.redblobgames.com/pathfinding/a-star/introduction.html>

[9] IEEE Robotics and Automation Society, "Introduction to Autonomous Mobile Robots," 2025.
[Online]. Available: <https://www.ieee-ras.org/>